

AN OPEN, AI-NATIVE RISC-V
CORE

一款开放的、原生 AI 的 RISC-V 内核

CrossV

越界 · Cross the Boundary
越界 · 跨越边界

A silicon for AI to learn from, design from, build on.
一块供人工智能学习、设计、构建的硅基芯片。

CROSSV · APR 2026
CROSSV · 2026 年 4
月

APACHE
2.0

V
0.1

The singularity
is not a moment.

It's a fork.

奇点并非一个瞬间。它是一个分叉。

The next CPU
won't be designed
by humans alone.

下一代CPU将不再仅由人类设计。

If AI will design silicon, it needs a silicon to learn from.

如果人工智能将设计芯片，它需要一个可供学习的芯片。

— CROSSV · OPEN CORE MANIFESTO — — CROSSV · 开源核心宣言

Three roads diverge here.

三条道路在此分岔。 AI、芯片设计、开源

奇点之前 · 三岔路口

Three roads, one vertex. 三条道路，一个顶点。

多数项目只选一条路。CrossV 站在三条路的交点 —— 这是我们的坐标。

01 · SILICON 01 · 硅晶

Silicon that ships. 可量产
量产的硅芯片。

可量产的芯片

对标 ARM Cortex-M7 的首颗 core。能跑 FPGA，能流到
流到 28 nm，能真正被用起来。

02 · AUTONOMY 02 · 自主性

Designed by AI. 由人工智
工智能设计。

AI 全自动设计

设计、验证、固件、手册，全流程 AI 驱动。人类只定
目标，其他交给模型。

03 · OPEN STACK 03 · 开放栈

Radically open. 彻底开放

。

RISC-V 全栈透明

ISA · RTL · testbench · bitstream · SDK · 编译器 · 文档
—— 一切 Apache 2.0。指令集架构 · 寄存器传输级 ·
测试平台 · 比特流 · 软件开发工具包 · 编译器 · 文档
—— 一切基于 Apache 2.0 协议。

Silicon is still stuck in 1998. 硅芯片技术仍停留在 1998 年。

AI 每 18 个月性能翻倍。芯片设计流程几乎没变。

TODAY 今日

Closed, slow, expensive.
封闭、缓慢、昂贵。

- 数百工程师 · 18-36 个月 tape-out 周期
- RTL、验证、固件、文档彼此脱节
- IP 被 EDA 厂与 arch license 绑死
- 每次迭代耗费千万美元

TOMORROW · CROSSV 明日 · CROSSV

Open, autonomous, cheap.
开放、自主、廉价。

- 一个人 + 一组 agent 在 FPGA 闭环
- 设计 / 验证 / 固件 / 手册 单一 pipeline
- ISA 无授权费 · RTL Apache 2.0
- 迭代按天计，不再按季度计

Why now. 为何是现在

三条独立的曲线在 2026 年第一次对齐 —— 这是 CrossV 存在的理由。

10¹⁸

TRAINING FLOPS · 2026 训练算力 · 2026

大模型已能稳定输出正确可综合的 RTL

ratified 已
批准

RISC-V PROFILES RISC-V 配置文件

RVA23 / RVM23 冻结 · 指令集成熟

<\$1k

FPGA + TOOLCHAIN FPGA + 工具链

开源综合 + 低成本板卡普及

0

APACHE-2.0 AI-NATIVE CORES APACHE-2.0
2.0 人工智能原生核心

此刻没有人把三者合起来做

02 · THE PIPELINE 02 · 流程管道

Prompt to silicon. 从提示到硅片

四段式 AI 流水线

STAGE 01 · DESIGN 第一阶段 · 设计

Design. 设计

AI 生成微架构 · 从 prompt 到 RTL

INPUT 输入

ISA profile · PPA budget · 目标应用 domain 指令集架构配置 · 功耗性能面积预算 · 目标目标应用领域

AGENTS 智能体系统

Arch planner → RTL writer → Lint & synth checker, 循环迭代 架构规划 → 寄存器传输级器传输级设计 → 语法检查与综合验证, 循环迭代

OUTPUT 输出结果

Verilog RTL + micro-architecture spec + 可综合网表 Verilog RTL + 微架构规范 + 可综合网 + 可综合网表

HUMAN LOOP 人工介入环节

只在 spec 阶段审阅 · RTL 之后全自动

CLAUDE · GPT · QWEN · LOCAL LLM Claude
· GPT · Qwen · 本地 LLM

CHIPYARD · CHISEL · AMARANTH

YOSYS · VERILATOR · SLANG

SURELOG · OPENFPGA

CROSSV AGENT RUNTIME CROSSV 代理运行时

Verify. 验证。

FPGA 自动验证 · 从仿真到真实硅片

TESTBENCH 测试平台

AI 生成 UVM-lite + 定向用例 · 覆盖率驱动回归

FORMAL 正式验证

SymbiYosys + RISC-V Formal ISA spec 比对 SymbiYosys + RISC-V 形式化指令集架构规范比
集架构规范比对

FPGA LOOP FPGA 循环

Xilinx Kintex / Lattice ECP5 夜间自动回归 · 失败自动 bisect

METRICS 指标数据

IPC · 覆盖率 · 面积 · 功耗 · 每次 commit 都留档

RISC-V FORMAL / RISCOF RISC-V 形式
化验证 / RISCOF 框架

SYMBIYOSYS · JASPER-LITE SYMBIYOSYS 形式验证工具 ·
具 · JASPER-LITE 验证平台

VERILATOR · COCOTB VERILATOR 仿真器 · COCOTB 协同仿真框架

XILINX VIVADO · OPENXC7 XILINX VIVADO 开发套件 ·
套件 · OPENXC7 开源工具链

LATTICE NEXTPNR

CROSSV BENCH-FARM CROSSV 基准测试场

STAGE 03 · SHIP 阶段 03 · 交付

Ship. 交付。

软件栈 · 从 bring-up 到应用

TOOLCHAIN 工具链

GCC / LLVM tuple · binutils · newlib · picolibc GCC / LLVM 工具链 · binutils · newlib · newlib · picolibc

RUNTIME 运行时

HAL · CMSIS-equivalent · FreeRTOS / Zephyr port 硬件抽象层 · 类 CMSIS 接口 · FreeRTOS/Zephyr 移植

EXAMPLES 示例程序

Blinky · UART echo · DSP FIR · 小型 TinyML 推理 LED 闪烁 · UART 回显 · DSP 有限冲激响应冲激响应滤波器 · 微型 TinyML 推理

DEBUG 调试

OpenOCD · JTAG dongle 配置 · GDB script · VSCode 模板 OpenOCD · JTAG 调试器配置 ·

LLVM · GCC · RUST

ZEPHYR · FREERTOS

OPENOCD · PROBE-RS

PLATFORMIO · RENODE

TFLITE-MICRO · TINYENGINE

Document. 文档。

说明书自书 · 与代码同源

AUTO-DOC 自动文档

从 RTL + 寄存器表 + testbench 自动生成数据手册

FORMATS 格式

Markdown 源 · 输出 HTML · PDF · IP-XACT

LIVING SPEC 动态规范

每次 commit 自动 rebuild · 与 RTL 永不脱钩

ASSISTANT 助手

内嵌 AI QA · 用户可直接问芯片

CROSSV.DEV
V

CROSSV DOCGEN CROSSV 文档生成器

MKDOCS · ASCIIDOC

IP-XACT · SYSTEMRDL

TYPST · PANDOC

RAG · EMBEDDED ASSISTANT 检索
增强生成 · 嵌入式助手

Prompt → Silicon. 从提示到硅片。

四段式 AI 流水线 · 每一步的产物都完全开源



STAGE 01 第一阶段

Design 设计

AI 生成微架构

从 ISA + budget 生成 RTL。输出 Verilog 与 spec。



STAGE 02 第二阶段

Verify 验证

FPGA 自动验证

生成 testbench, formal, coverage。FPGA 回归闭环。生成测试平台、形式验证、覆盖率。FPGA 回归测试闭环。



STAGE 03 阶段 03

Ship 发布

SDK 与固件

GCC / LLVM tuple · HAL · 示例工程 · OpenOCD。
GCC / LLVM 工具链 · 硬件抽象层 · 示例项目 ·
OpenOCD。



STAGE 04 阶段 04

Document 文档

说明书自书

数据手册 · 用户指南 · 白皮书。与代码同 commit。

A vertex to start from.
一个全新的开始。
这是一个开源项目

先跑通 · 再加速

03 · FIRST CORE 03 · 首款核心

Meet Vertex. 遇见 Vertex。

顶点 · 家族的第一颗

ISA

RV32IMAFC + Bv

FPU / DSP 浮点运算单元 / 数字信号处理器

Single · 16-bit SIMD 单指令多数据 · 16 位

PERF 性能

2.14 DMIPS/MHz

PIPELINE 流水线

5-stage, in-order 5 级流水线, 顺序执行

CLOCK 时钟频率

480 MHz @ 28 nm

TARGET 目标

ARM Cortex-M7



Vertex M7 顶点 M7

CV-01 · FIRST SILICON

CV-01 · 首颗芯片

A topology of cores. 一套核心拓扑结构。

#	NAME 名称	TIER 等级	ROLE 角色	STATUS 状态
CV-01	Vertex M7 顶点 M7 顶点	~ARM M7 约等于 ARM M7	高性能 MCU · 旗舰起点	IN DEV 开发中
CV-02	Tangent M4 切线	~ARM M4	低功耗嵌入式	PLANNED 计划中
CV-03	Nexus A55 枢纽	~ARM A55	64-bit · Linux 应用处理器	PLANNED 计划中
CV-04	Manifold X1 流形 X1 流形	AI ACC AI 加速器	RVV 1.0 · 矩阵扩展	PLANNED 计划中
CV-05	Meridian M0 子午线	~ARM M0+ 约等于 ARM M0+	极简 · IoT · 电池供电	PLANNED 计划中

Ship the environment first. 优先部署环境

目标不是最快的 core，而是最快能跑通的全流程。先闭环，再优化。

QUARTER 季度节点	MILESTONE 里程碑节点	DELIVERABLE 可交付成果	STATUS 状态
Q2 · 2026 2026 年第二季度	Environment 环境	AI pipeline 闭环 · FPGA bring-up · Blinky on real hardware AI 流水线闭环 · FPGA 上板调试 · 实体硬件跑马灯测试测试	IN PROGRESS 进行中
Q3 · 2026 2026 年第三季度	Vertex M7 α	RV32IMAFC 通过 RISC-V compliance · CoreMark 上板 RV32IMAFC 通过 RISC-V 兼容性测试 · CoreMark 上板实测板实测	NEXT 下一步
Q4 · 2026 2026 年第四季度	SDK 1.0	GCC/LLVM tuple · Zephyr port · 12 个示例工程 GCC/LLVM 工具链 · Zephyr 移植 · 12 个示例项目	NEXT 下一步
Q1 · 2027 2027 年第一季度	Tape-out RC 流片准备就绪	28 nm shuttle 申请 · DRC/LVS clean · datasheet 1.0 28 纳米流片申请 · DRC/LVS 验证通过 · 数据手册 1.0 版	LATER 稍后

Everything. In the open. 一切。皆在开放中。

从 ISA 到 datasheet · 一切 Apache 2.0 · 没有隐藏层

What's open. 开放了什么。

RTL VERILOG · CHISEL	Testbench 测试平台 UVM-LITE · COCOTB COCOTB
FPGA bitstream FPGA 比特流 KINTEX · ECP5 ECP5	SDK 软件开发工具包 GCC · LLVM
Runtime 运行时 ZEPHYR · FREERTOS	Datasheet 数据手册 MD · PDF · IP-XACT
AI Agents AI 智能体 PROMPTS · TOOLS 提示词 · 工具	Benchmarks 性能基准 LOGS · TRACES 日志 · 追踪记录

How we govern. 我们的治理方式。

Apache 2.0 · 无专利陷阱 · 贡献者协议对等。所有 agent prompt、训练曲线、FPGA 回归日志与志与芯片 bring-up 视频全部公开。

社区治理：RFC 流程 · 公开 RFC 仓库 · 每周 tech sync 直播 · 年度 CrossV 峰会。

不存在的东西：闭源 EDA 依赖 · 付费 arch license · 商业早鸟版本。

Fork the **future.** 开创未来。

加入开源社区 · 共建 AI 自己的 CPU

* github.com/crossv

CROSSV · TEAM
DECK

Team.

团 队 简 介

Six people, one RISC-V core.

Two **initiators.** One thesis: open silicon, **built together.**

两位项目发起人 · 一个共同的判断：开放的硅，应当由开放的协作共建。

3 full-time · 3 part-time · one shared thesis.

— WU WEI · WU YE · AND THE BUILDERS

Project Initiators.

项目发起人 · 软件 × 系统



INITIATOR · SOFTWARE

吴伟
WU WEI

苦芽科技联合创始人 · LLVM 基金会理事 · RISC-V 国际基金会技术委员会委员 · PLCT
PLCT 实验室创始人 · "甲辰计划"发起人。

ROLE

Software & Compilers
Compilers



INITIATOR · SYSTEMS

吴冶
WU YE

英特尔智能边缘计算联合研究院常务副院长 · 20+ 年软硬件全栈系统专家 · 主导
5G ORAN、TD-LTE 小基站、100G 以太网等系统 · 英特尔最高荣誉 IAA。

ROLE

Full-stack Systems / CEO

The full stack, covered.

3 全职 + 3 兼职 · 覆盖从编译器到 FPGA 验证的完整链路 3 full-time + 3 part-time · compilers → architecture → MCU → FPGA → verification

01 · INITIATOR · 兼职

吴伟 WU WEI

苦芽科技联合创始人 · LLVM 基金会理事 · PLCT 创始人 · "甲辰计划"发起人

Software & Compilers

02 · INITIATOR · CEO · 全职

吴冶 WU YE

英特尔智能边缘计算联合研究院常务副院长 · 20+ 年软硬件全栈 · IAA

Full-stack Systems

03 · ARCHITECT · 全职

李东声 LI DONGSHENG

英特尔处理器架构师 · CPU 大核 / NPU / SoC · 专利 10+ · 两本架构著作

CPU Architecture

04 · HARDWARE · 全职

陈曦 CHEN XI

英特尔硬件设计专家 · 高性能计算 10+ 年 · 架构 → 原型全流程

MCU Design

05 · VERIFICATION ·

石侃 SHI KAN · PhD

中科院计算所副研究员 · RISC-V 国际基金会技委委员 · Imperial PhD · MICRO/ASPLOS/HPCA

FPGA Verification

06 · VERIFICATION · 兼职

穆洋 MU YANG

博瑞晶芯 UT 负责人 · 13 年前端验证 · ARM V9 / 香山 RISC-V CPU 核心验证

Front-end EDA

INITIATOR · SOFTWARE & COMPILERS · PART-TIME

吴伟

WU WEI
WEI

苦芽科技联合创始人 · CrossV 项目发起人 · 负责项目软件和编译器

在开源编译器和 RISC-V 社区有多年深耕。2019 年创建并
并领导 [PLCT 实验室](#)，推动全球 RISC-V 软件生态的发展，
展，在 [RT-Thread on RISC-V](#) 方面有长期投入。2024 年除
年除夕，联合多家机构共同发起 "[甲辰计划](#)" ——致力于
在 2036 年前实现 RISC-V 软件生态的全面繁荣。

CURRENT

苦芽科技 联合创始人

APPOINTMENTS

LLVM 基金会 [理事](#) RISC-V 国际基金会 [技术委员会委员](#) CCF
CCF 系统软件专委 RISC-V 基础软件工作组 [组长](#)

LEGACY

PLCT 实验室 创始人 (2019) 甲辰计划 联合发起人 (2024)



INITIATOR · FULL-STACK SYSTEMS / CEO · FULL-TIME

吴治

WU YE
YE

英特尔智能边缘计算联合研究院 常务副院长 · 公司 CEO · 全职项目发起人

20+ 年 软硬件全栈系统专家，具备从芯片到软件、再到复杂系统的全链条整合，以及 AI 驱动系统构建与商业化落地 的顶尖能力。曾主导过定义行业标准的系统，包括高性能 RISC-V CPU、5G 车联网芯片、虚拟化基站系统（5G ORAN 基石）、首、首个 TD-LTE 小型基站、100G 以太网芯片系统等。

CURRENT

英特尔智能边缘计算联合研究院 常务副院长 CrossV CEO
CEO

TRACK RECORD

高性能 RISC-V CPU 5G 车联网芯片 · 5G ORAN TD-LTE 小基
LTE 小基站（行业首个） 100G 以太网芯片系统

HONOR

英特尔最高荣誉 IAA — Intel Achievement Award



LEAD · CPU ARCHITECTURE · FULL-TIME

李东声

LI DONGSHENG
DONGSHENG

英特尔智能边缘计算联合研究院 处理器架构师 · 全职负责高性能处理器架构设计



专注于高性能处理器架构设计与关键技术研究。参与和主导多项高性能 CPU 大核与 NPU 架构以及 SoC 设计；商用产品覆盖服务器、终端、边缘计算等多个领域。授权国内外发明专利十余项。

FOCUS

高性能 CPU 大核 · NPU · SoC

PATENTS

国内外发明专利 10+

AUTHORED

《高性能超标量 CPU 微架构剖析与设计》《AI 处理器硬件架构设计》

LEAD · MCU DESIGN · FULL-TIME

陈曦

CHEN XI

英特尔智能边缘计算联合研究院 硬件设计专家 · 全职负责项目 MCU 设计

深耕高性能计算领域十余年，擅长将复杂算法转化为极致的硬件架构。具有从 芯片架构设计到原型实现 的全流程项目经验。

FOCUS

MCU 设计 · 硬件架构

EXPERIENCE

高性能计算 10+ 年 算法 → 硬件架构

FULL-CYCLE

芯片架构设计 · 原型实现 · 全流程



LEAD · FPGA VERIFICATION · PART-TIME

石侃

PHD

SHI KAN

中科院计算所 副研究员 · RISC-V 国际基金会 技术委员会委员 · 负责项目 FPGA 验证

研究方向：芯片敏捷设计与验证、FPGA、云数据中心体系结构等。博士毕业于伦敦帝国理工学院，曾任英特尔英国研发中心芯片设计工程师，兼任英特尔研究院研究科学家。有超过十年的芯片研究和从业经历，研究成果发表于 MICRO、ASPLOS、HPCA、FPGA 等国际顶会。

CURRENT

中科院计算所 副研究员 RISC-V 国际基金会 技术委员会委员

EDUCATION

Imperial College London · PhD

AWARDS

IEEE Micro Top-Picks 论文奖 HiPEAC 论文奖 × 2 · 最佳论文提名文提名 × 1



LEAD · FRONT-END EDA VERIFICATION · PART-TIME

穆洋

MU YANG
YANG

博瑞晶芯 UT 验证负责人 · 负责项目前端 EDA 验证 (兼职)

拥有 13 年前端验证经验、6 年以上处理器与 SoC 验证经验，
，专注于 ARM 与 RISC-V 架构及高性能处理器验证。主导 ARM
ARM V9 系列及香山 RISC-V 系列 CPU 核心验证项目，覆盖 CPU
CPU 核心完整功能验证路径，具备验证架构规划、自动化平台
台建设与团队带领经验。

CURRENT

博瑞晶芯 UT 验证负责人

EXPERIENCE

前端验证 13 年 · 处理器 / SoC 6+ 年 ARM · RISC-V · 高性能处理
能处理器

LED PROJECTS

ARM V9 系列 CPU 核心验证 香山 RISC-V 系列 CPU 核心验
心验证 参与 6 款千万级量产芯片验证



● A I - N A T I V E H A R D W A R E V E R I F I C A T I O N L O O P

AI 原生硬件验证闭环 · V 2.0

AI × Verif

驱 动 · 闭 环 · 签 收

Drive · Close the loop · Sign off.

Three weeks.

三周 ~ 几月 —— 验证工程师的真实代价。

01

TESTPOINT

测试点分解

3-5 天 ~ 1-2 个月

- 维度不一致，corner case 漏了无人察觉
- 跨模块方法论全靠经验，无法复用
- 8 维还是 12 维？谁来保证不遗漏？

02

UVM ENV

环境搭建

3 ~ 7 天

- agent 分组靠人判断，RM 建模靠摸索
- 宏依赖链一断，编译报错来回调试
- >100 端口大模块，wrapper 写到手抽筋

03

SIGNOFF

覆盖率收敛 → 核签

2-4 周 ~ 4-5 月

- 218 条 checklist 人肉逐条走
- waiver 谁来判、谁来签、依据是什么？
- 核签包拼凑 6 份文档，重复劳动

Six skills, two paths,

六个 skill，两条路径，一个签收。

Hardware Verification Skill Pipeline

6 skills · 两条验证闭环 · 从 RTL+Spec 到人工签收



Testpoint Gen - 如何选择

- testpoint-gen (v1) - 普通模块, 6-phase, 8维度, 16项质量检查, 够用就选
- testpoint-gen-v2 - 推荐新模块: 12维度, RTL inventory密度校验, cross-pollinator覆盖验证, 21项检查

UVM Env Gen - 如何选择

- uvm-env-gen (v1) - 普通模块: 自动 RTL 分析 - agent 分箱, 6-phase, 输出完整 UVM 目录树
- uvm-env-gen-v2 - 复杂子系统: >5子模块或>100端口时 Phase 1 自动生成 wrapper, 宏/参数宽度解析

verify-signoff-driver - 关键说明

- 两条闭环共享同一个 skill, 输入 testpoints.json + 可编译 UVM env 即可启动
- 6个 stage 状态机: Baseline → RefModel → Sequences → Assertions → Regression → Sign-off

Pick your road.

两条路径，共同终点。

01

REGULAR MODULE PATH

普通模块路径

RTL + Spec 输入材料
testpoint-gen v1 / v2 8 或 12 维度分解
uvm-env-gen v1 / v2 可编译 UVM 环境
verif-signoff-driver 驱动到 sign-off

适用：接口清晰、无ISA 解码、<5 子模块
MUYANG.DEV

02

ISA / COMPLEX PATH

ISA / 复杂模块路径

RTL + Spec + ISA Spec 输入材料
isa-coverage-mapper ★ 自动抽取 ISA 表 · 双源互校
testpoint-gen-v2 12 维度 + ISA 子树合并
uvm-env-gen-v2 auto-wrapper · 大模块支持
verif-signoff-driver 驱动到 sign-off (同路径①)

适用：decode / ISA 模块、≥5 子模块、>100 端口

From guess to formula.

从 "靠经验" 到 "用公式"。

v1

testpoint-gen

6 - PHASE FLOW

6-phase 流程, Phase 1 × 4 并行 agent 读材料

8 FUNCTIONAL DIMS

8 功能维度: 结构 / 输入 / 状态 / 更新 / 输出 / 恢复 / 时序 / cross

16 QUALITY CHECKS

16 项质量检查, 失败可 re-dispatch

3 - FORMAT OUTPUT

三格式输出: testpoints.md / .xlsx / .json

ISA OPTIONAL

ISA 输入可选; 普通模块不需要

v2

testpoint-gen ★

7 - PHASE FLOW

新增 Phase 1.5 Inventory + Phase 3.5 Cross-Pollinator

12 DIMENSIONS (+4)

8 功能 + 4 非功能 (Power / DFT / Security / Perf)

21 QUALITY CHECKS

新增 inventory 命中率、非功能比例校验

INVENTORY-DRIVEN

TP 目标按 RTL 复杂度算公式, 拒绝拍脑袋

DECODE AUTO-DELEGATE

Phase 3b 委托 isa-coverage-mapper, 合并 ISA 子树

Inventory drives everything.

RTL Inventory 决定一切。

- 0

INPUT CHECK

输入检查
- 1

PARALLEL READ

并行读材料 ×4
- 1.5

RTL INVENTORY

枚举信号/FSM/RAM
- 2

CROSS-SYNTHESIS

12 维功能树 + 追溯
- 3

12-DIM PARALLEL

12 子 Agent 并发
- 3.5

CROSS-POLLINATOR

扫漏网条目 · 兜底
- 4

QUALITY + OUTPUT

21 项检查 + 三格式

MUYANG.DEV
21 QUALITY CHECKS = 6 critical (fail → restart) + 15 supplementary + 4 ICVS-1.1 new

TP_TARGET DENSITY FORMULA

Phase 1.5 — RTL 复杂度 → 测点目标数

```
TP_target = max(10,  
  0.4 × [signal_count / 20]  
  + 3 × fsm_count  
  + 2 × ram_count  
  + 0.8 × config_bit_count  
  + 0.2 × LoC / 1000)
```

12 DIMENSIONS · 8 functional + 4 non-functional

D1 结构 D2 输入 D3 状态 D4 更新 D5 输出 D6 异常 D7 时序 D8 Cross

D9 Power · D10 DFT · D11 Security · D12 Perf

非功能维度占比须在15%~35% (质检#21)

Dual-source ISA table.

RTL + Spec 双源自动提取。

TRIGGER · 触发条件

module 含 decode / idecode / rv_dec / fp_alu / issue / dispatch / rename — 或任何需要 ISA encoding 覆盖的模块



OUTPUTS · 四个交付物

testpoints_isa.json / .xlsx

fcov_isa.sv (standalone + UVM)

conflict_report.md

traceability_matrix.csv

One id, two agents.

同一个 enc_id，两个 Agent 独立提取。

ENC_ID DUAL-SOURCE ALIGNMENT

enc_id = SHA256(canonical_encoding_template)[:6]

RTL Agent A

解析 case / casez / ROM
→ 独立生成 enc_id

SPEC Agent B

识别 PDF 编码表
→ 独立生成 enc_id

↓ 相同算法 → enc_id 一致 → Phase 2 精确 join

CANONICAL ISA ENTRY · 23 FIELDS

14 BASE FIELDS · 必填

enc_id · mnem · isa_family · category · encoding_template · variable_fields
rtl_decode_signal ← Agent A · behavior_summary · spec_section ← Agent B

9 SUPPLEMENTARY · 补充字段

implicit_operands · special_reg(ZR/SP) · imm_range · privilege_level · hint_flag ...
故意留空字段 → 两 Agent 互不污染

T1~T8 CONFLICTS · Phase 2

T1 encoding_templatecritical	T2 field_definitioncritical
T3 operand_typecritical	T4 behaviornormal
T5 dependencynormal	T6 hint_attributenormal
T7 aliasnormal	T8 unallocatedminor

R1~R6 ARBITRATION

R3 编码冲突 → 信 RTL · R4 行为冲突 → 信 Spec · R5 矛盾 → 升级用户

PHASE 4a · 21 SCENARIO CODES

S1~S9	通用	Normal · 特殊寄存器 · 立即数边界 · 特权级 · 非法编码
F1~F5	FP 浮点	舍入模式 · NaN · 无穷 · 非规格化数 · 异常标志
M1~M4	MEM 内存	对齐 · 页跨越 · 原子类型 · 内存顺序
B1~B3	分支	跳转/不跳 · BPU 预测 · 间接跳转

Auto-wrapper for the big ones.

大模块也能自动封装。

v1

uvm-env-gen

6-PHASE FLOW

RTL 分析 → 架构设计 → 代码 → 编译 → RM → 文档

AUTO AGENT GROUPING

按端口前缀 / 协议自动归组

RM PATTERNS

behavioral / table-based / hybrid, 含 skip-flag

AUTO COMPILE FIX

max 10 轮自动 fix, VCS 编译就绪

FCOV GENERATION

传 testpoints.json 自动生成 fcov.sv

v2

uvm-env-gen ★

AUTO-WRAPPER

≥ 5 子模块或 > 100 端口 → gen_wrapper.py 自动生成

RTL MACRO TRACE

Phase 1 自动 trace `define 链, 字面整数写入 SV

SUBSYSTEM UT

无 parent module 场景强制 v2 wrapper, 纯子系统验证

8-DIM EVALUATION

Phase 2 评接口清晰度 / FSM / 时钟域 → 适宜度评分

FCOV_ISA INTEGRATE

可选接入 isa-coverage-mapper 输出的 fcov_isa.sv

Lessons baked in.

踩过的坑，下次不用再踩。

- Ph 1

RTL ANALYSIS
宏递归展开 · wrapper 触发判断 · clk/FSM 识别
- Ph 2

ARCH DESIGN ★
端口分组 6 步 → agent → RM → fcov · 用户确认
- Ph 3

WRAPPER (on demand)
≥5 子模块或 >100 端口触发
- Ph 4

CODEGEN ★★
agent / env / tb / tc / common · 5 并行 Agent
- Ph 5

COMPILE LOOP
10 轮迭代 · 14 种错误 pattern 自动修复
- Ph 6

RM REFINE LOOP
tc_random + SCB enabled 通过为止

PHASE 4 · FIVE PARALLEL AGENTS

Agent-A	ALL UVM AGENTS	cfg / xaction / sqr / drv / mon / cov / if / pkg
Agent-B	ENV LAYER	cfg / rm / scb / vseqr / virtual_seq / env_pkg
Agent-C	TB LAYER	top_tb / dut_inst / connect / clk_rst_if
Agent-D	TC LAYER	base / sanity / random / debug / corner / error
Agent-E	COMMON + SIM	xaction / fcov / filelist / Makefile / regress.py

34 HARD RULES · 来自 踩坑

- R1 Interface 硬编码宽度，禁用 RTL 宏

R2 Sequencer 用 uvm_component_utils
- R4 SCB 用 \$isunknown() 选择性比较

R5 RM 内置 3 个 skip flag
- R8 tc_sanity 关 SCB · tc_random 开 SCB

R13 Makefile 带 -cm line+cond+fsm+tgl+branch

Six stages, five checkpoints.

六阶段，五个人工检查点。

路径① 与 路径② 在此汇聚 · 同一个 skill



ITLB 实测 · 2026-04-09 → 04-12

FUNC COV	LINE COV	BRANCH	CONDITION	TC PASS	OPEN BUGS	REGRESSION	TOTAL TIME
100%	99.86%	99.74%	92.86%	17 / 17	0	1000+ seed	2.5 days

Classify, then close.

分类，然后收敛。

E1 配置 →	E2 触发 →	E3 解析 →	E4 更新DB →	E5 失败分类 →	E6 漏洞分析 →	E7 距离指标 →	E8 收敛判断
---------	---------	---------	-----------	-----------	-----------	-----------	---------

E5 REGRESSION FAILURES · 5 类

dut_bug	严重
DUT 违反 spec → bugs.jsonl	
tb_bug	中
tb 代码有误 → agent-tb-fixer 自动补丁	
seq_bug	中
sequence 生成非法激励 → 修 constraint	
refmodel_bug	● 最严重
RM 逻辑有误 → 升级回 Stage B 重新校准	
flaky	低
随机重现 → 标记 + 多次验证	

SIGNOFF CRITERIA · 默认

功能 100% · Line ≥ 95% · Branch ≥ 90% · ≥ 1000 seed · 连续 3 轮 clean · 0 critical/high bug

E6 COVERAGE GAPS · 5 类

unreachable
逻辑不可能命中 → 起草豁免 (含证明)
missing_stimulus
可达但缺激励 → 新增 sequence (回 Stage C)
illegal_bin
Bin 定义本身有问题 → 修改 fcov.sv
flaky
命中数波动大 → 调采样 · 增加种子
genuinely_hard
理论可达但概率极低 → 升级 · 专项 seed

Five judgments. The rest auto-runs.

五个判断，其余全自动。

1	Phase 2 · tp-gen	MODULE UNDERSTANDING	AI 读 RTL+Spec 综合后，人确认理解准不准
2	Stage B · signoff	RM SEMANTIC REVIEW	Reference Model 建模策略与 skip-flag 是否合理
3	Stage D · signoff	ASSERTION SET	SVA 方向对吗，有无明显遗漏
4	Stage E · signoff	WAIVER REASONABILITY	覆盖率 waiver 条目，逐条说明理由是否充分
5	Stage F · signoff	FINAL SIGN-OFF	6 份文档 · 218 条 checklist 审阅后亲笔签收

Same skill, four modules.

同一套 skill, 4 个真实模块。

ITLB	ICACHE	DECODE	LSU
✓ SIGNED OFF	TP DONE	STATIC SIGNED	STAGE A-E DONE
TESTPOINTS S282	210	196&2700+isa	1445
FUNC COV100%	100%	96%*	—
LINE99.86%	99.17%	96%*	—
BRANCH99.74%	99.25%	91%*	—
TC PASS17/17	38/38	22 seq	8/8
OPEN BUGS0	0	0	0
SVA20	39	52	78
AGENTS6	8*	8	7
TIME2.5 d	4d	~22h	~23h

Four sentences to start.

四句话启动。

`isa-coverage-mapper`

"给 xx 模块做 ISA 覆盖"

decode / ISA 模块首选

`testpoint-gen v2`

"给 xx 模块做测试点分解"

全模块通用 · 支持断点续跑

`uvm-env-gen v2`

"给 xx 模块生成 UVM 验证环境"

支持断点续跑

`verif-signoff-driver`

"把 xx 模块推进到核签"

两条路径共用 · 支持断点续跑

PREREQUISITES · 你需要准备

RTL 路径 + Spec 文档 (decode 类额外加 ISA Spec)

ThankVou. 感谢。



越 界 · CROSS THE BOUNDARY 越 界 · 跨越边界